INSTITUTO TECNOLÓGICO DE VERACRUZ

Nombre de la Asignatura:

- Administraciòn en Base de Datos

Grupo: **AD26** **6J3A**

Nombres del alumno:

- Raúl Alberto Guerrero Aguilar
- Num. Control: 23600266

Nombre de la Actividad:

- Reporte de Práctica 8

Carrera: Ingeniería en Sistemas Computacionales

Fecha de Entrega: 23/Octubre/2026

1. Objetivo de la Práctica

Implementar, configurar y verificar espacios de tablas (*General Tablespaces* y *File-Per-Table Tablespaces*) en el motor de almacenamiento InnoDB dentro de un contenedor MySQL gestionado con Podman en Linux Mint, documentando la resolución de problemas de conexión por socket vs. red TCP/IP, el manejo de la consola interactiva y las consideraciones de persistencia en entornos contenedorizados.

---

2. Marco Teórico: Arquitectura de Tablespaces en MySQL (InnoDB)

Un **tablespace** (espacio de tablas) en MySQL permite almacenar los datos de forma separada del archivo general del sistema por defecto (`ibdata1`) o de los archivos individuales por defecto de cada base de datos, lo cual resulta fundamental para optimizar el rendimiento, organizar el almacenamiento físico y facilitar las tareas de administración.

2.1. Tipos de Tablespaces en InnoDB

- **System Tablespace (`ibdata1`):** Es el espacio de tablas predeterminado del sistema donde InnoDB guarda el diccionario de datos interno y los registros de control transaccional (*undo logs*).
- **General Tablespace (Tablespace General):** Es un espacio de tablas compartido creado explícitamente por el administrador de base de datos mediante la instrucción `CREATE TABLESPACE`. MySQL almacena estos espacios en archivos físicos con extensión `.ibd` dentro de su directorio de datos por defecto (`/var/lib/mysql/`) y permite asociar múltiples tablas a un mismo archivo `.ibd`.
- **File-Per-Table Tablespace (`innodb_file_per_table`):** Por defecto, las versiones modernas de MySQL tienen habilitada la variable `innodb_file_per_table = ON`, lo que significa que cada tabla creada genera automáticamente su propio archivo `.ibd` independiente dentro de la carpeta correspondiente a su base de datos.

---

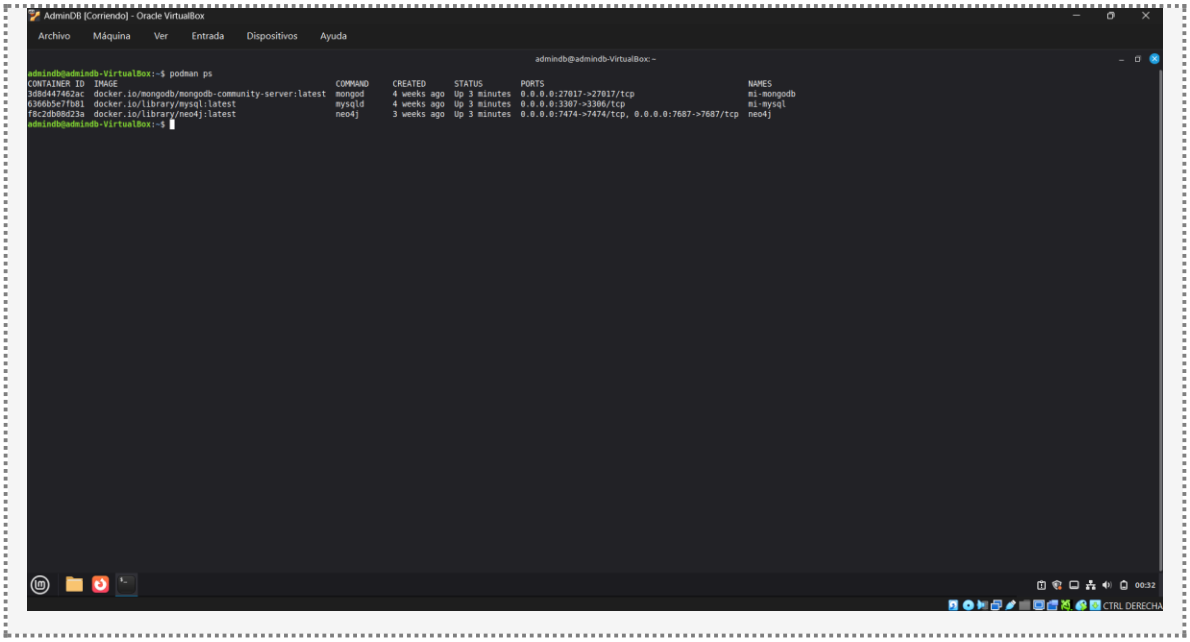
3. Reconocimiento del Entorno en Podman

Antes de ingresar a MySQL, se verificaron los contenedores activos dentro de la terminal de Linux Mint (`admindb@admindb-VirtualBox`) utilizando el comando de listado de procesos de Podman:

```
podman ps
```

El entorno cuenta con tres motores de bases de datos contenerizados en ejecución:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3d8d447462ac	docker.io/mongodb/mongodb-community-server:latest	mongod	4 weeks ago	Up 3 minutes	0.0.0.0:27017->27017/tcp	mi-mongodb
6366b5e7fb81	docker.io/library/mysql:latest	mysqld	4 weeks ago	Up 3 minutes	0.0.0.0:3307->3306/tcp	mi-mysql
f8c2db08d23a	docker.io/library/neo4j:latest	neo4j	3 weeks ago	Up 3 minutes	0.0.0.0:7474->7474/tcp, 0.0.0.0:7687->7687/tcp	neo4j



Evidencia 1: Captura de pantalla inicial en Oracle VirtualBox mostrando la ejecución de `podman ps` y la identificación del contenedor objetivo `mi-mysql` (`6366b5e7fb81`)

---

4. Desarrollo Paso a Paso de la Práctica

Paso 1: Acceso al Contenedor de MySQL y Solución del Error de Socket (ERROR 2002)

1.1. Intento de conexión inicial (Fallo por Socket Unix)

Conociendo que el nombre del contenedor es `mi-mysql`, se intentó acceder inicialmente a la consola del cliente MySQL ejecutando:

```
podman exec -it mi-mysql mysql -u root -p
```

Al introducir la contraseña de `root`, el contenedor devolvió el siguiente error:

```
Enter password:
ERROR 2002 (HY000): Can't connect to local MySQL server through socket
'/var/run/mysqld/mysqld.sock' (2)
```

1.2. Explicación técnica del error y solución mediante TCP/IP

Este error ocurre porque al invocar `mysql -u root -p` directamente con `podman exec` sin especificar un host de red, el cliente de MySQL intenta buscar el servidor localmente a través de un archivo de socket Unix en la ruta `/var/run/mysqld/mysqld.sock` en lugar de usar la red TCP/IP. Al correr dentro de un contenedor aislado donde el socket no se encuentra en esa ruta por defecto para dicha invocación, la conexión es rechazada.

Para solucionarlo, se debe indicar explícitamente al cliente que se conecte mediante **red TCP/IP** utilizando la bandera `-h 127.0.0.1`:

```
podman exec -it mi-mysql mysql -h 127.0.0.1 -u root -p
```

Al ejecutar este comando e ingresar la contraseña, el monitor de MySQL inició sesión correctamente:

```
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 26.7.0 MySQL Community Server - GPL
```

Paso 2: Creación de un Tablespace General (General Tablespace)

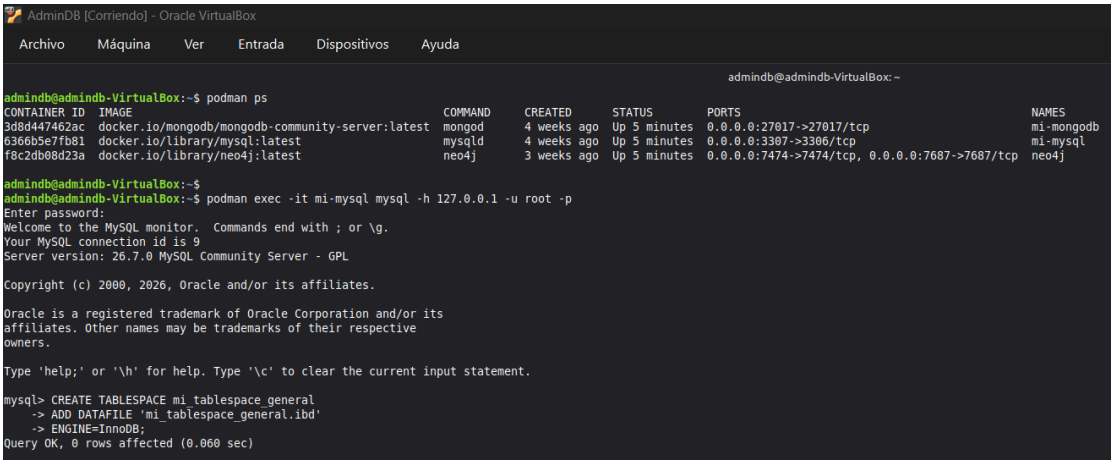
Una vez dentro de la consola `mysql>`, se creó un tablespace general llamado `mi_tablespace_general` asociado al archivo de datos `mi_tablespace_general.ibd` bajo el motor `InnoDB`:

```
CREATE TABLESPACE mi_tablespace_general
ADD DATAFILE 'mi_tablespace_general.ibd'
ENGINE=InnoDB;
```

Resultado en consola:

```
Query OK, 0 rows affected (0.060 sec)
```

- **Nota sobre la ruta de almacenamiento:** Por defecto, cuando solo se especifica el nombre del archivo `'mi_tablespace_general.ibd'`, MySQL lo crea directamente dentro de su directorio principal de datos (`/var/lib/mysql/`) dentro del contenedor.



Evidencia 2: Captura de pantalla donde se observa el ingreso exitoso con `podman exec -it mi-mysql mysql -h 127.0.0.1 -u root -p` y la creación del tablespace general con resultado `Query OK, 0 rows affected (0.060 sec)`

Paso 3: Comandos utilitarios para explorar esquemas y limpiar la terminal

Para administrar adecuadamente la sesión interactiva de MySQL y la terminal de Linux Mint se emplean los siguientes comandos:

- **Ver todas las bases de datos disponibles en el servidor** (tales como `information_schema`, `mysql`, `performance_schema`, `sys` y las creadas por el usuario):

```
SHOW DATABASES;
```

- **Crear una base de datos nueva (en caso de no contar con una):**

```
CREATE DATABASE mi_base;
```

- **Seleccionar la base de datos de trabajo:**

```
USE nombre_de_tu_base_de_datos;
```

- **Listar las tablas dentro de la base de datos seleccionada:**

```
SHOW TABLES;
```

- **Limpiar la pantalla sin salir de la consola de MySQL (`mysql>`):**

```
\! clear
```

(Este comando invoca el `clear` del sistema operativo anfitrión/contenedor sin cerrar la sesión de MySQL).

- **Limpiar la pantalla desde la terminal normal de Linux Mint (`admindb@admindb-VirtualBox:~$`):**

```
clear
```

*(O bien utilizando el atajo de teclado **`Ctrl + L`**).*

Paso 4: Creación de una Tabla Asociada al Tablespace General

Una vez limpiada la consola y verificadas las bases de datos existentes, se seleccionó la base de datos **`tienda`** y se creó la tabla **`empleados`** asociándola explícitamente a `mi_tablespace_general` mediante la cláusula `TABLESPACE`:

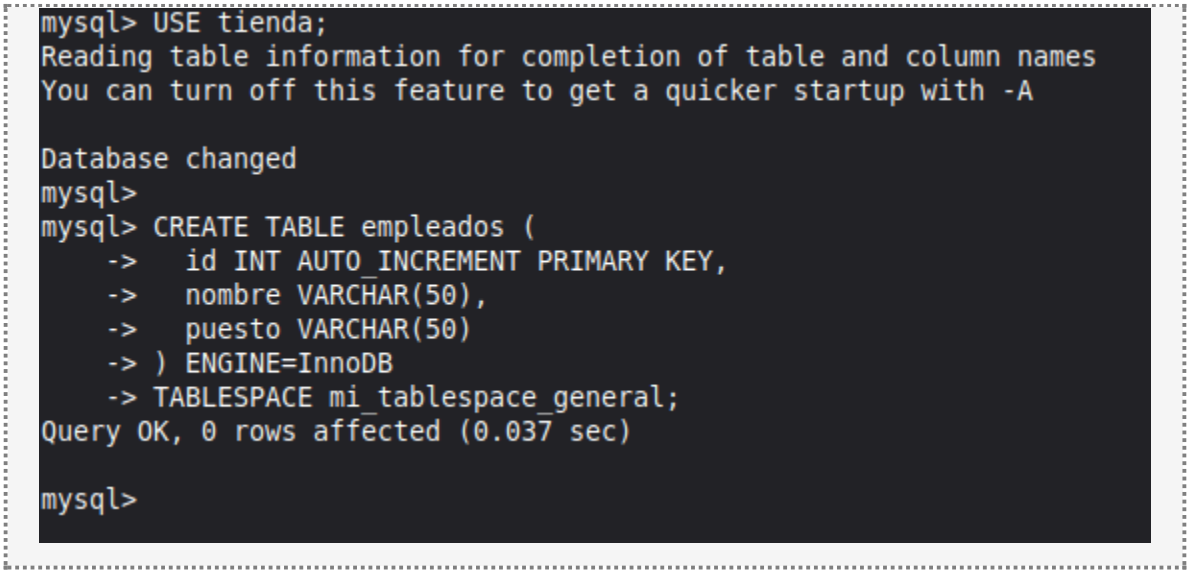
```
USE tienda;

CREATE TABLE empleados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50),
    puesto VARCHAR(50)
) ENGINE=InnoDB
TABLESPACE mi_tablespace_general;
```

Resultado en consola:

```
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
Query OK, 0 rows affected (0.037 sec)
```



Evidencia 3: Captura de pantalla mostrando la selección exitosa de `USE tienda;` (`Database changed`) y la ejecución del `CREATE TABLE empleados` vinculado a `TABLESPACE mi_tablespace_general` en `0.037 sec`

Paso 5: Alternativa — Crear una Tabla con su Propio Tablespace Independiente (File-Per-Table)

Aunque MySQL moderno ya tiene activada por defecto la opción `innodb_file_per_table = ON` (donde cada tabla crea automáticamente su propio archivo `.ibd` dentro de la carpeta de su base de datos), también es posible declarar explícitamente que una tabla específica utilice su propio tablespace individual con la cláusula `TABLESPACE = innodb_file_per_table`:

```
CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    empresa VARCHAR(100)
) ENGINE=InnoDB
TABLESPACE = innodb_file_per_table;
```

Paso 6: Verificación y Comprobación en el Diccionario de Datos

Para comprobar en la consola de MySQL que tanto el tablespace general como la tabla `empleados` quedaron registrados y vinculados correctamente, se ejecutan las siguientes consultas:

6.1. Verificar que el tablespace general se creó en InnoDB

```
SELECT *
FROM INFORMATION_SCHEMA.INNODB_TABLESPACES
WHERE NAME LIKE '%mi_tablespace_general%';
```

6.2. Confirmar que la tabla empleados utiliza mi_tablespace_general

```
SELECT TABLE_NAME, TABLESPACE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'tienda';
```

6.3. Listar las tablas existentes dentro de la base de datos tienda

```
SHOW TABLES;
```

---

5. Consideraciones Arquitectónicas al Usar Tablespaces en Podman

Al trabajar con bases de datos MySQL dentro de contenedores administrados con **Podman**, es indispensable tener en cuenta dos aspectos críticos de infraestructura:

1. **Persistencia de Datos (Volúmenes):** Dado que MySQL se ejecuta dentro del contenedor `mi-mysql`, si el contenedor llega a eliminarse sin contar con un volumen (`-v` o `--volume`) mapeado hacia el sistema anfitrión (Linux Mint), se perderán tanto las bases de datos como los archivos `.ibd` de los tablespaces creados. Es fundamental asegurar que `/var/lib/mysql` esté respaldado por un volumen persistente en Podman.
2. **Ubicaciones Personalizadas y Permisos de Usuario (UID):** Si en lugar de usar la ruta por defecto se requiere que el archivo `.ibd` del tablespace resida en una ruta externa específica dentro de Linux Mint mapeada hacia el contenedor, se debe verificar que el directorio posea los permisos de lectura y escritura correspondientes para el usuario interno con el que corre el proceso `mysqld` dentro del contenedor (generalmente el usuario `mysql`, con UID 999 o equivalente en el mapeo de subuid/subgid de Podman).

---

6. Conclusión

La práctica permitió implementar de forma completa el ciclo de vida de un *General Tablespace* (`mi_tablespace_general`) en MySQL `26.7.0` corriendo sobre Podman. Se comprobó cómo diagnosticar y resolver el error `ERROR 2002 (HY000)` derivado de la búsqueda de sockets Unix locales dentro de contenedores mediante el uso explícito de conexiones TCP/IP (`-h 127.0.0.1`), así como la vinculación física de la tabla `empleados` en el esquema `tienda`, diferenciando claramente entre el almacenamiento compartido de un tablespace general y el enfoque individual de `innodb_file_per_table`.